

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Product Name

EKSAT

On-Demand Local Service Platform

1. Product Overview

EKSAT is an on-demand service marketplace platform that connects customers with verified local service providers for household, personal, and support services.

The platform allows:

- **Customers** to discover and request services
- **Service Providers** to register, get approved, receive job requests, and complete work
- **City Admins** to manage providers, monitor jobs, and oversee operations at city level
- **Super Admins** to manage the entire platform across cities and territories

The system is designed to streamline the full service lifecycle from:

provider onboarding → customer request → provider assignment → service execution → payment → reporting

This PRD is based on the client's shared project concept and role definitions, including service provider, customer, city admin, and super admin modules, along with the described workflow and feature direction.

2. Problem Statement

Customers often struggle to find **trusted, available, and nearby service providers** for everyday work such as plumbing, electrical work, security support, pet care, cleaning, and similar local services.

At the same time, many skilled workers lack a structured digital platform where they can:

- register their skills
- receive verified work opportunities
- manage jobs professionally
- track work and earnings

Current workflows are often fragmented across:

- phone calls
- WhatsApp
- manual referrals
- local agents
- paper-based coordination

This creates issues such as:

- unreliable service matching
- low trust
- poor job tracking
- payment confusion

- no structured admin oversight

EKSAT aims to solve this by creating a centralized service platform with operational control, provider verification, customer convenience, and administrative visibility.

3. Product Vision

To build a trusted and scalable local service ecosystem where customers can easily access skilled workers and providers can receive structured job opportunities through a monitored and efficient digital workflow.

4. Product Goals

The platform should:

1. Allow service providers to register and offer services professionally
2. Allow customers to quickly request local services
3. Ensure providers are verified and manageable through admin controls
4. Enable smooth service request, assignment, and completion workflows
5. Track job execution and operational performance
6. Support payment and payout visibility
7. Create a scalable admin-controlled platform for city-wise expansion

5. Success Criteria

The first version of the product should successfully support:

- Provider registration and approval
- Customer service request creation
- Provider job acceptance/rejection
- Service completion tracking
- Basic payment / payout tracking
- Admin oversight of jobs and providers

Initial success indicators

- Providers can complete end-to-end jobs through the app
- Customers can request and track services without manual intervention
- Admin can monitor and manage all active requests
- Core job lifecycle is stable and traceable

6. Target Users

6.1 Customers

Individuals or households who need local service assistance.

Examples:

- someone needing a plumber
- someone needing pet sitting
- someone needing a security guard
- someone needing home repair

6.2 Service Providers

Individuals offering local service-based work.

Examples:

- plumber
- electrician
- cleaner
- pet watcher / pet sitter
- painter
- helper
- security worker
- appliance technician

6.3 City Admin

An operational manager responsible for managing service activity within a city or service territory.

6.4 Super Admin

The platform-level administrator responsible for managing all cities, admins, permissions, analytics, and system-wide operations.

7. User Roles and Responsibilities

7.1 Customer

A customer should be able to:

- create an account / log in
- browse available service categories
- create a service request
- provide service details and location
- view request status
- see job history
- re-book past services
- receive notifications related to service progress

7.2 Service Provider

A provider should be able to:

- register as a service provider
- create and manage profile
- choose one or more skill categories
- upload KYC / identity / verification documents
- set service availability
- receive service request notifications
- accept or reject jobs
- start and end work timer
- view active and completed jobs

- track basic earnings / payout info

These requirements align with the provider-focused features in the client document such as profile management, OTP login, skill category selection, availability toggle, job handling, push notifications, accept/reject flow, work timer, live tracking, and wallet/earnings concepts.

7.3 City Admin

A city admin should be able to:

- view and manage providers within the city
- approve or reject provider registrations
- view and manage customer accounts
- monitor all active and completed jobs
- assign providers to requests (if required by workflow)
- track payments and settlements
- access basic operational reports
- view category and provider performance

This is supported by the city admin features described in the client document, including provider approval, account management, monitoring, tracking, and reporting expectations.

7.4 Super Admin

A super admin should be able to:

- manage all cities and city admins
- manage permissions and role access
- view overall platform metrics
- oversee system-wide settings and operational controls
- access advanced reporting and administrative visibility

This aligns with the document's super admin and state-level platform control direction.

8. Product Scope

The product includes three main interfaces:

8.1 Customer App / Interface

Used by customers to request and track services

8.2 Provider App / Interface

Used by providers to manage work and receive jobs

8.3 Admin Panel

Used by city admins and super admins for management and oversight

9. Core Product Flow

The intended end-to-end product flow is:

1. Provider registers on the platform
2. Admin reviews and approves provider
3. Customer browses service categories
4. Customer creates a service request

5. Matching / eligible provider(s) are notified
 6. Provider accepts or rejects the job
 7. Provider travels to customer location
 8. Provider starts service timer
 9. Provider completes work and ends timer
 10. Job is marked completed
 11. Invoice / payment record is generated
 12. Customer feedback is collected
 13. Admin dashboard reflects updated operational data
- This flow is directly based on the workflow and process sections in the client document.

10. Functional Requirements

10.1 Authentication & Access

Customer

- Customer should be able to register and log in securely
- OTP-based login should be supported for mobile-first access
- Customer should be able to manage profile details

Provider

- Provider should be able to register and log in securely
- OTP-based login should be supported
- Provider access should remain restricted until admin approval

Admin

- City Admin and Super Admin should have secure admin login
- Admin access should be role-based

10.2 Provider Onboarding

The provider onboarding flow should include:

- account creation
- profile setup
- contact information
- skill/service category selection
- city / service area selection
- optional work experience information
- KYC / ID document upload
- availability setup
- application submission for approval

Admin actions

Admin should be able to:

- review provider applications
- approve or reject provider registration
- suspend or deactivate providers if needed

10.3 Provider Profile Management

A provider should be able to manage:

- name
- profile image
- phone number
- address / location
- skill categories
- service area
- work availability
- experience details
- verification documents (if editable under rules)

10.4 Service Categories

The system should support configurable service categories.

Example categories:

- plumbing
- electrical work
- security
- pet care
- cleaning
- painting
- home maintenance
- appliance repair

Admin should be able to:

- create category
- edit category
- enable/disable category

10.5 Customer Service Discovery

Customers should be able to:

- browse services by category
- search by keyword
- discover relevant service types
- optionally view available providers (if enabled in product flow)

The client document references customer-side service search and multi-category browsing.

10.6 Service Request Creation

A customer should be able to create a service request with the following:

- service category
- job title / short summary
- detailed job description
- service location / address

- preferred date and time
- contact details
- optional image upload (future-ready but not mandatory for MVP)

Each request should create a trackable job record in the system.

10.7 Job Matching / Assignment

This is one of the most important modules in the platform.

When a customer creates a request, the system should support assigning that request to eligible providers.

MVP recommended behavior

For MVP, assignment should follow a **controlled dispatch model**:

Recommended MVP flow:

- request is created
- matching providers are identified based on skill and city / service area
- admin can view candidate providers
- provider is assigned / notified
- provider accepts or rejects

This approach is safer and more manageable for the first version than fully automated dispatch.

Future-ready direction

The platform may later support:

- nearby provider broadcasting
- smart allocation
- first-come-first-served acceptance
- geo-based matching

The client document references nearby provider notifications, request casting, geo-fencing, and smart allocation ideas, but these should be treated as later-stage or controlled MVP variants unless explicitly prioritized.

10.8 Provider Job Handling

Once a job is assigned or offered, the provider should be able to:

- view job details
- accept job
- reject job
- see scheduled jobs
- see immediate / active jobs
- view customer location / contact info (as allowed)
- view current status

The provider module in the source document explicitly includes job handling, instant notifications, accept/reject flow, scheduling, and tracking expectations.

10.9 Job Status Lifecycle

The platform should support a clearly defined job lifecycle.

Recommended job statuses

- REQUESTED

- UNDER_REVIEW
- ASSIGNED
- PENDING_PROVIDER_RESPONSE
- ACCEPTED
- REJECTED
- EN_ROUTE
- IN_PROGRESS
- COMPLETED
- CANCELLED

Rules

- only eligible state transitions should be allowed
- invalid transitions should be blocked
- admins should have override capabilities where necessary

This job lifecycle will become one of the most important backend state machines in the system.

10.10 Work Tracking

The platform should support provider-side work tracking.

A provider should be able to:

- mark arrival / start work
- start service timer
- end service timer
- mark work as completed

System should store:

- start time
- end time
- duration
- completion timestamp

The source document specifically references start/end timer and work tracking in the provider module and workflow.

10.11 Location Tracking

The system should support provider location visibility to the extent required by the product.

MVP

- store provider city / service area
- optionally capture job location
- optionally record provider's live work location if implemented in controlled form

Future

- live map tracking
- geofencing
- ward mapping
- route optimization

The client document includes live tracking, geo-fencing, and ward mapping ideas, but these should be scoped carefully.

10.12 Notifications

The system should send notifications for important workflow events.

Examples:

- provider registration approved/rejected
- new job assigned
- provider accepted/rejected job
- job started
- job completed
- payment recorded
- status changes

Notification channels

MVP may support:

- in-app notifications
- push notifications

SMS / WhatsApp / email can be added later if needed.

10.13 Customer Job Tracking

Customers should be able to:

- see current job status
- see assigned provider info (if enabled)
- view service history
- re-book previous services

This is consistent with the customer module and workflow expectations in the source document.

10.14 Payments, Invoice, and Payout Visibility

The system should support basic payment and payout tracking.

MVP recommendation

Start with **manual / admin-assisted payment tracking**, not full payment automation.

Customer-side

- payment status should be visible

Admin-side

- admin should be able to mark payment state
- admin should be able to view provider payout state

Provider-side

- provider should be able to see basic payout / earning records

Payment states

Recommended:

- UNPAID
- PAYMENT_PENDING

- PAID
- PAYOUT_PENDING
- PAYOUT_COMPLETED

Invoice

The system should create a basic invoice or service completion record after job completion.

The client document references invoice generation, payment processing, wallet concepts, finance management, settlement management, and commission-related reporting.

10.15 Feedback & Ratings

After a completed service, the customer should be able to:

- provide rating
- leave optional feedback

MVP

- simple star rating + optional text feedback

Future

- provider ranking
- trust scoring
- moderation logic

The client document references rating and feedback collection in the workflow.

10.16 Admin Panel Features

10.16.1 Provider Management

Admin should be able to:

- view all providers
- filter by status / city / skill
- approve/reject providers
- suspend or deactivate providers

10.16.2 Customer Management

Admin should be able to:

- view customer accounts
- manage customer access / support cases

10.16.3 Job Monitoring

Admin should be able to:

- view all service requests
- filter by status / city / category / provider
- view job details
- assign or reassign providers
- monitor ongoing jobs
- review completed jobs

10.16.4 Payment & Settlement Monitoring

Admin should be able to:

- track customer payment status
- track provider payout status
- view service financial records
- review settlement state

10.16.5 Reporting

Admin should be able to access:

- daily / monthly job reports
- category demand reports
- provider performance reports
- operational summaries

The client document includes several admin-side reporting and control expectations, which should be implemented in phased depth.

10.17 Super Admin Features

The super admin should be able to:

- manage all cities / territories
- manage city admins
- configure role access
- view high-level platform metrics
- access advanced reporting
- manage platform-wide operational controls

Future / internal-only platform functions

Some super admin functions in the source document such as database monitoring, API monitoring, system logs, deployment controls, and version-related operational visibility may be relevant later, but they should not all be treated as product MVP requirements.

11. Business Rules

This section is critical.

Provider Rules

- Provider must be approved before receiving jobs
- Suspended or rejected providers must not receive new jobs
- Provider can only act on jobs assigned or offered to them
- Provider should not edit restricted job data after completion

Job Rules

- A job must belong to exactly one customer
- A job must belong to one service category
- A job can have only one active assigned provider in MVP
- A rejected job should return to the assignment queue

- A completed job should be locked from normal editing
- Job completion and payment should be tracked separately

Payment Rules

- Payment state must be separate from job state
- Provider payout state must be tracked independently
- Invoice / completion record should be generated after completion

Admin Rules

- Only authorized admin roles can approve providers
- Only admin or permitted system logic can assign providers
- Super admin has broader operational access than city admin

12. Non-Functional Requirements

12.1 Security

The system should support:

- secure authentication
- protected role-based access
- secure admin routes
- secure provider document handling
- audit visibility for sensitive actions where relevant

The source document references API security, JWT-style concepts, audit logs, and compliance-related expectations.

12.2 Performance

The system should support stable usage for early-stage operational rollout.

MVP should focus on:

- reliable request creation
- reliable assignment flow
- responsive dashboard interactions
- consistent notification delivery

12.3 Scalability

The platform should be designed in a way that can later support:

- multiple cities
- more providers
- more categories
- territory-based operations
- future dispatch automation

12.4 Maintainability

The system should be structured modularly so that:

- features can be extended
- workflows can evolve

- future payment / dispatch automation can be added without major rewrite

13. MVP Scope

This is the most important scoping section.

13.1 In Scope for MVP

Customer

- OTP login / basic account access
- browse service categories
- create service request
- view request status
- job history
- re-booking

Provider

- register / login
- profile setup
- skill selection
- availability toggle
- KYC submission
- receive job notification
- accept / reject job
- start / end work timer
- view assigned jobs

Admin

- approve/reject providers
- manage providers and customers
- view all jobs
- assign / reassign provider
- monitor job status
- basic payment / payout tracking
- basic reporting

Platform

- notifications
- core job lifecycle
- basic invoice / completion record
- feedback collection

This MVP direction is consistent with the strongest functional parts of the source document while intentionally avoiding overbuilding.

13.2 Out of Scope for MVP (Later Phase)

These are valid future features, but should not block v1:

- dynamic pricing engine
- fraud detection engine

- escalation matrix automation
- smart dispatch algorithm
- first-come-first-served auto-assignment
- geofencing
- ward mapping
- advanced live map operations
- multilingual support
- wallet system
- automated commission engine
- automated GST / local tax logic
- advanced analytics
- API monitoring dashboard
- system deployment controls
- enterprise-scale observability tools

Many of these appear in the client document as platform ambitions or advanced ideas, but they should be handled as roadmap features, not MVP blockers.

14. Assumptions

The following assumptions are being made for the first version:

1. Provider onboarding requires admin approval
2. OTP is the primary login approach for customer and provider access
3. Customer request flow is city / service-area aware
4. Provider assignment in MVP is controlled and not fully automated
5. Only one provider will actively own a job at a time in MVP
6. Payment automation is not required in MVP
7. Ratings and feedback are simple in MVP
8. Multi-city support is needed structurally, even if rollout starts with one city

15. Open Questions / Clarifications Required

These must be confirmed before HLD and LLD are finalized.

Assignment Logic

1. Should the customer choose a specific provider, or only request a service category?
2. Should providers receive jobs automatically based on location, or should admin approve assignments first?
3. If multiple providers are notified, who gets the job?

Scheduling

4. Should jobs support both immediate and scheduled bookings?
5. Can a provider handle overlapping bookings?

Payment

6. Will customers pay online, offline, or both?
7. Does the platform take a commission from provider earnings?
8. Should provider payouts be manual or automated?

Provider Rules

9. Can one provider register under multiple skills?
10. What verification documents are mandatory?
11. Can providers update profile after approval?

Customer Experience

12. Should customers be able to see provider ratings before booking?
13. Should customers be able to cancel after provider accepts?

Operations

14. Should city admins only see data for their city?
 15. Should super admin have access to all city-level operations?
 16. Is live location tracking mandatory for MVP or optional?
- These questions are essential because they directly affect system design and workflow logic.

16. Recommended Product Strategy

To maximize delivery success, the product should be built in layers:

Phase 1 — Core Marketplace Engine (MVP)

Build the operational heart of the system:

- onboarding
- request creation
- provider dispatch
- job completion
- payment visibility
- admin control

Phase 2 — Operational Optimization

Add:

- better matching
- analytics
- smarter dispatch
- better provider performance tracking

Phase 3 — Platform Intelligence & Scale

Add:

- pricing logic
- fraud control

- territory optimization
- advanced automation
- deeper finance and reporting systems

17. Recommended Next Deliverables After This PRD

This PRD should now be used to create the next technical documents:

Next step documents

1. **HLD (High Level Design)**
2. **LLD (Low Level Design)**
3. **Screen / Page List**
4. **Database Entity Design**
5. **API Route Plan**
6. **Admin Permission Matrix**
7. **MVP Timeline / Build Plan**

Final Summary

EKSAT is a multi-role, admin-governed, on-demand local service platform designed to connect customers with verified service providers through a controlled service request and execution workflow.

The product's core value lies in making the following flow reliable:

Request → Match / Assign → Accept → Work → Complete → Pay → Track

That is the heart of the system.

Everything else is secondary.